ChatGPT said:

Diwali Sky – Personalized Fireworks Greeting Web Application

A. Project details

- Page slug: `diwali-sky`
- Category: Software Development
- Level: Expert
- Time to build: 22 hrs
- Card summary (max 100 characters): Create personalized fireworks greetings using JavaScript, Canvas Physics and Web Audio.
- Page subtitle (one sentence, max 160 characters): Turn a person's name into a spectacular fireworks display using real-time particle physics, procedural sound and privacy-first web engineering.
- Related program: Wrench Wise – Software Development Track (Frontend Engineering, JavaScript and Creative Web Technologies)

B. Guide content

1. Objective

Build **Diwali Sky**, an interactive web application that transforms a recipient's name into a spectacular fireworks display over an illuminated South Indian temple skyline. You will engineer a real-time graphics pipeline using HTML5 Canvas, JavaScript physics, particle pooling, typography rasterization, procedural Web Audio synthesis and privacy-preserving URL sharing. This project demonstrates advanced frontend engineering, memory optimization, mathematical modelling and browser API integration, making it a substantial portfolio project beyond conventional CRUD applications.

2. Prerequisites

- **HTML5 and CSS3:** Understand semantic HTML, forms, responsive layouts, CSS positioning, gradients, animations and CSS custom properties.
- **JavaScript ES6+:** Be comfortable with classes, objects, arrays, modules (`import` / `export`), callbacks, destructuring and event listeners.
- **Core Mathematics:** Understand coordinate systems, Cartesian geometry, vectors, velocity, acceleration, gravity, trigonometry and interpolation.
- **Data Structures:** Understand arrays, typed arrays, indexing, object pooling and basic time and space complexity.
- **Browser APIs:** Have introductory knowledge of Canvas 2D, DOM manipulation, `requestAnimationFrame`, Web Audio and browser storage.
- **Development Environment:** Install a modern browser (Chrome or Edge), Visual Studio Code, Git and Python 3. Node.js is optional.

3. Tools you will use

- **HTML5:** Build the semantic application structure and interactive canvas elements.
- **CSS3:** Implement responsive layouts, layered visual effects, gradients, transitions and animation controls.
- **JavaScript (ES2022+):** Develop the animation engine, physics calculations, particle management, audio synthesis and application logic.
- **Visual Studio Code:** Organize modules, implement functionality, debug errors and inspect source code.
- **Chrome DevTools:** Monitor FPS, inspect JavaScript execution, debug Canvas rendering, analyze browser performance and execute automated tests.
- **Python 3:** Run a local HTTP server for testing JavaScript ES modules.
- **Git and GitHub:** Track source-code changes and maintain the project repository.
- **Vercel:** Deploy the application as a static website with automatic production delivery.

4. Steps

Step 1: Initialize the Project Structure and Responsive Interface

- **Instruction:**

Establish the application foundation before implementing animation logic.

1. Create a project directory named `Diwali_Sky`.
2. Organize the project into three primary directories:
   - `assets/` – Store the logo, favicon and social-sharing preview image.
   - `js/` – Store JavaScript modules responsible for application functionality.
   - `tools/` – Store development utilities and Open Graph image generation tools.
3. Create `index.html` as the application entry point.
4. Create `styles.css` and define reusable CSS variables for the brand palette, typography, backgrounds, borders and shadows.
5. Design the initial interface with:
   - Wrench Wise branding.
   - A dark nighttime background.
   - A responsive greeting form.

- Recipient and sender name fields.
- Happy Diwali and Happy Deepavali selection buttons.
- A primary Light it up button.

6. Add a mobile viewport configuration with `viewport-fit=cover`.
7. Use `100dvh` for the application height to accommodate dynamic mobile browser controls.

Recommended initial structure:

Plain text

```
Diwali_Sky/
│
├── assets/
│   ├── logo.png
│   ├── favicon.png
│   └── og-image.png
│
├── js/
│   ├── main.js
│   └── ui.js
│
├── tools/
│   └── og.html
│
├── index.html
├── styles.css
├── manifest.json
└── LICENSE
```

Architecture decision: Separate presentation from application logic. HTML handles structure, CSS handles appearance, and JavaScript modules handle behaviour. This makes individual features easier to develop, test and maintain.

- - Screenshot (optional): `diwali-sky-step-1.png`
- - Checkpoint: The application loads successfully on desktop and mobile, displays the complete greeting form, and has no HTML or CSS errors in DevTools.

Step 2: Construct the Multi-Canvas Rendering Architecture

- **Instruction:**

Build the graphics foundation using multiple HTML5 Canvas elements instead of drawing every visual element on a single canvas.

1. Add three canvas elements to `index.html`:
   - `#sky-canvas` – Background atmosphere, stars, skyline, temple and lanterns.
   - `#fireworks-canvas` – Rockets, particles, sparks and animation effects.
   - `#bloom-canvas` – Additional glow and illumination effects.
2. Create `js/sky.js` and implement a `SkyManager` class.
3. Capture the browser viewport dimensions using `window.innerWidth` and `window.innerHeight`.
4. Calculate the rendering resolution using `devicePixelRatio`, capped at 2.0.
5. Implement canvas resizing and context scaling.
6. Attach `resize` and `orientationchange` event listeners.
7. Ensure that all canvases remain aligned when the device orientation changes.

Canvas architecture:

Plain text

```
Browser Viewport
      |
      v
  SkyManager
      |
      +----------------------+
      |          |           |
      v          v           v
  Sky Canvas  Fireworks   Bloom Canvas
      |        Canvas          |
```

```
      |          |             |
      v          v             v
  Background  Particles       Glow
  Architecture Rockets       Effects
      |          |             |
      +----------+-------------+
                 |
                 v
           Final Visual Scene
```

Resolution calculation:

JavaScript

```
const pixelRatio = Math.min(window.devicePixelRatio || 1, 2);

canvas.width = window.innerWidth * pixelRatio;
canvas.height = window.innerHeight * pixelRatio;

canvas.style.width = `${window.innerWidth}px`;
canvas.style.height = `${window.innerHeight}px`;

context.setTransform(pixelRatio, 0, 0, pixelRatio, 0, 0);
```

Engineering decision: Multiple canvases isolate rendering responsibilities. Background architecture does not need to be recalculated whenever fireworks particles move. This reduces unnecessary drawing operations and makes independent visual optimization possible.

- • Screenshot (optional): `diwali-sky-step-2.png`
- • Checkpoint: All three canvases render at the correct viewport dimensions, remain aligned after resizing and orientation changes, and display no scaling or clipping errors.

Step 3: Implement the High-Performance Particle Management System

- **Instruction:**

Develop a particle engine capable of managing thousands of simultaneously active particles without repeatedly allocating and destroying JavaScript objects.

1. Create `js/particles.js`.
2. Implement a `ParticlePool` class.
3. Pre-allocate memory for 7,000 particles using typed arrays.
4. Store particle attributes in parallel buffers:
   - Position: `x`, `y`.
   - Velocity: `vx`, `vy`.
   - Lifetime and size.
   - Particle type and color identifiers.
5. Use `Float32Array` for numerical properties and `Uint8Array` for compact identifiers.
6. Maintain an `activeIndices` array to track active particles.
7. Implement particle spawning, updating, rendering and deactivation.
8. When a particle expires, use swap-and-pop removal rather than shifting the entire active array.
9. Implement gravity, air resistance and particle lifetime calculations.

Particle memory architecture:

JavaScript

```
const MAX_PARTICLES = 7000;

const positionsX = new Float32Array(MAX_PARTICLES);
const positionsY = new Float32Array(MAX_PARTICLES);

const velocityX = new Float32Array(MAX_PARTICLES);
const velocityY = new Float32Array(MAX_PARTICLES);

const lifetimes = new Float32Array(MAX_PARTICLES);
const sizes = new Float32Array(MAX_PARTICLES);
```

```
const colors = new Uint8Array(MAX_PARTICLES);

let activeCount = 0;
```

Particle removal using swap-and-pop:

JavaScript

```
function removeParticle(index) {
    activeCount--;

    const lastIndex = activeIndices[activeCount];

    activeIndices[index] = lastIndex;
    activeIndices[activeCount] = index;
}
```

Important optimization:

A conventional implementation might create thousands of new particle objects during every explosion. This increases garbage collection pressure.

Object pooling avoids that repeated allocation. Particle memory is reserved once and reused throughout the animation.

Complexity analysis:

Operation	Complexity
Particle spawning	O(1)
Particle removal	O(1)
Particle updating	O(N)
Particle rendering	O(N)

- - Screenshot (optional): `diwali-sky-step-3.png`
- - Checkpoint: The particle pool initializes with a capacity of 7,000, supports particle creation and removal, never exceeds its configured capacity, and reuses allocated memory without creating new particle objects for each frame.

Step 4: Engineer Rocket Physics and Firework Burst Algorithms

- **Instruction:**

Implement the mathematical behaviour behind rocket launches and different fireworks formations.

1. Create `js/fireworks.js`.
2. Implement a `FireworksEngine` class.
3. Calculate rocket trajectories using initial velocity, acceleration and elapsed time.
4. Ensure each rocket reaches its target position at the specified launch duration.
5. Implement four firework formations:
    - **Peony:** Radial explosion with randomly distributed particle velocities.
    - **Ring:** Uniformly distributed particles forming a circular shape.
    - **Willow:** Slowly descending golden sparks with longer lifetimes.
    - **Crackle:** Two-stage explosions where dying particles generate smaller secondary particles.
6. Apply gravity and air resistance independently to each formation.
7. Connect rocket launches and explosions to the particle management system.

Rocket trajectory mathematics:

Given:

- Starting position: (x_0, y_0)
- Target position: (x_1, y_1)
- Launch duration: t

Calculate initial vertical velocity:

\[
v_{y0}=\frac{2(y_1-y_0)}{t}
\]

Calculate vertical acceleration:

\[
a_y=-\frac{2(y_1-y_0)}{t^2}
\]

Calculate horizontal velocity:

\[
v_x=\frac{x_1-x_0}{t}
\]

Example implementation:

JavaScript

```
function calculateRocketPhysics(start, target, duration) {

    const deltaX = target.x - start.x;
    const deltaY = target.y - start.y;

    const velocityY = (2 * deltaY) / duration;
    const accelerationY = -velocityY / duration;

    const velocityX = deltaX / duration;

    return {
        velocityX,
        velocityY,
        accelerationY
    };
}
```

Ring formation mathematics:

$$\theta_i = \frac{2\pi i}{N} + \theta_{rotation}$$

$$v_x = v\cos(\theta_i)$$

$$v_y = v\sin(\theta_i)$$

Engineering decision: Separate burst algorithms into independent behaviours rather than implementing every explosion through one generic random distribution. This allows each firework to have its own visual identity, physical parameters and performance characteristics.

- • Screenshot (optional): `diwali-sky-step-4.png`
- • Checkpoint: All four firework types generate their intended formations, rockets reach their calculated target positions within the configured duration, and particle motion responds correctly to gravity and drag.

Step 5: Develop the Procedural Web Audio Synthesizer

- **Instruction:**

Generate realistic firework sounds directly through browser audio mathematics without downloading or storing prerecorded sound files.

1. Create `js/audio.js`.
2. Implement a `SoundManager` class using the Web Audio API.
3. Initialize an `AudioContext` after an explicit user interaction.
4. Create a `DynamicsCompressorNode` and connect the audio processing chain to the destination.
5. Implement four sound-generation methods:
    - `playLaunch()` – Rising-frequency sine wave for rocket launches.

- `playBurst()` – Low-frequency sine thump combined with filtered noise.
- `playCrackle()` – Multiple short band-pass filtered noise bursts.
- `playTempleBell()` – Four simultaneous sine oscillators using bronze bell frequency ratios.

6. Use gain envelopes and exponential frequency ramps to produce natural sound decay.
7. Restrict simultaneous audio voices to eight.
8. Implement mute and unmute functionality using master gain.
9. Store the mute preference in browser `localStorage` using the key `ds-muted`.

Audio architecture:

```
Plain text
User Interaction
       |
       v
   AudioContext
       |
       v
 Sound Generators
       |
       +----------------+
       |       |        |
       v       v        v
    Launch  Explosion  Crackle
       |
       v
 Dynamics Compressor
       |
       v
 Audio Destination
       |
       v
    Speakers
```

Temple bell synthesis frequencies:

Oscillator	Frequency	Gain
Fundamental	480 Hz	0.12
Tier 2	1324.8 Hz	0.07
Tier 3	2592 Hz	0.035
Tier 4	4272 Hz	0.015

Engineering decision: Procedural audio eliminates the need for large audio assets and provides precise synchronization between visual events and sound effects. The compressor also helps control audio peaks when multiple effects overlap.

- • Screenshot (optional): `diwali-sky-step-5.png`
- • Checkpoint: Rocket launches produce rising whistles, explosions produce low-frequency booms, crackles generate short noise bursts, the temple bell produces four-tone synthesis, and the audio engine never exceeds eight simultaneous voices.

Step 6: Implement Typography Rasterization and Name-Forming Physics

- **Instruction:**

This is the central engineering challenge of the project. Convert ordinary text into thousands of coordinates so that individual fireworks particles can assemble into readable letters.

1. Create `js/sprites.js`.
2. Generate 24 pre-rendered glow sprites:
   - Seven core colour sprites.
   - Sixteen gradient sprites.
   - One gold glint sprite.
3. Create `js/nameText.js`.
4. Render the recipient's name onto an offscreen canvas using a bold system sans-serif font.
5. Automatically adjust the font size between 26px and 140px.

6. Restrict the name to 86% of the viewport width and 30% of its height.
7. Read the canvas pixel data using `getImageData()`.
8. Identify solid text pixels using an alpha threshold greater than 120.
9. Calculate the sampling interval based on the target particle budget.
10. Apply controlled random jitter to avoid a rigid pixel-grid appearance.
11. Create `js/show.js` and implement the seven-phase animation timeline.
12. Make the explosion particles move outward and then home toward their assigned text coordinates.
13. Hold the completed name for 3.5 seconds before initiating the golden melt.

Typography sampling formula:

\[
S=\max\left(1,\operatorname{round}
\sqrt{\frac{4P_{solid}}{P_{target}}}\right)
\]

Where:

- S = Sampling interval.
- P_{solid} = Number of solid text pixels.
- P_{target} = Desired number of particles.

Random jitter:

\[
jitterX=(2R-1)(0.45S)
\]

\[
jitterY=(2R-1)(0.45S)
\]

Here, R is a random number between 0 and 1.

Name formation timeline:

Plain text

```
0.0s       Opening fireworks
2.5s       Hero rocket launches
3.5s       Hero explosion
4.05s      Particle homing begins
5.2s       Final staggered arrivals
6.0s       Name hold begins
7.75s      Thumbnail captured
9.5s       Golden melt begins
12.7s      Ambient mode
```

Gradient architecture:

The name uses three brand colours:

- Emerald: `#34D399`
- Teal: `#2DD4BF`
- Cyan: `#22D3EE`

The normalized horizontal coordinate determines the appropriate gradient sprite:

$$index = \text{round}(u \times 15)$$

where u ranges from 0 to 1.

Performance optimization: Once all name particles reach their target positions, combine them into a single offscreen image using `bakeHoldCanvas()`. During the hold phase, render the complete name with one `drawImage()` call instead of repeatedly drawing thousands of individual particles.

- Screenshot (optional): `diwali-sky-step-6.png`
- Checkpoint: Entering a recipient name produces readable text-shaped particle coordinates, the sparks assemble from left to right, the gradient transitions from emerald to cyan, the name remains visible for 3.5 seconds, and the golden melt begins at 9.5 seconds.

Step 7: Build the Animated Temple Skyline and Environmental Effects

- **Instruction:**

Create the visual environment that makes the fireworks display feel like a complete festival experience rather than an isolated particle animation.

1. Create `js/sceneLayers.js`.
2. Generate separate offscreen canvases for:
   - A distant city skyline containing 20 building silhouettes.
   - A seven-tier South Indian temple gopuram.
   - A foreground rooftop ledge with clay diyas.
3. Create `js/sceneRenderers.js`.
4. Implement animated stars, glowing windows, lanterns, mist and diya flames.
5. Create `js/scene.js` and implement `SceneManager`.
6. Add device tilt and mouse-based parallax movement.
7. Apply independent maximum horizontal shifts:
   - City: 4px.
   - Temple: 10px.
   - Foreground ledge: 18px.
8. Implement radial burst illumination to cast coloured light across the background.
9. Add the receiver illumination sequence.
10. Animate the city windows from left to right and illuminate the seven temple tiers from bottom to top.
11. Implement reduced-motion behaviour using `prefers-reduced-motion`.

Parallax architecture:

Plain text

```
User Mouse / Device Movement
            |
            v
       SceneManager
            |
       +-----+------+
       |     |      |
       v     v      v
      City Temple  Ledge
       4px  10px   18px
       |     |      |
       +-----+------+
            |
            v
       Composite Scene
```

Environmental engineering decision: Pre-render static architectural elements into offscreen canvases. Reuse those images during animation rather than recalculating complex architectural geometry every frame.

Accessibility requirement: When reduced motion is enabled, disable parallax and camera zoom, reduce burst particle counts, disable crackle sub-explosions and transition the name directly into its final formation.

- Screenshot (optional): `diwali-sky-step-7.png`
- Checkpoint: The skyline, seven-tier temple, diyas, lanterns and atmospheric effects render correctly; parallax respects the configured movement limits; the receiver scene completes its 1.5-second illumination sequence; and reduced-motion preferences disable the specified effects.

Step 8: Integrate the User Interface and Privacy-Preserving Sharing System

- **Instruction:**

Connect the animation engine with the sender and receiver experiences. Implement validation, private URL generation and sharing functionality.

1. Create `js/validate.js` and `js/blocklist.js`.
2. Validate recipient names between 1 and 16 characters.

3. Validate optional sender names up to 20 characters.
4. Handle Unicode letters and grapheme clusters using `Intl.Segmenter`.
5. Implement whitespace sanitization and blocked-word validation.
6. Create `js/link.js` and implement Base64url encoding and decoding.
7. Use a versioned JSON payload:

```javascript
const sharePayload = {
    v: 1,
    t: "Priya",
    f: "Vivek",
    g: "diwali"
};
```

8. Serialize the payload using `JSON.stringify()` and encode its UTF-8 representation.
9. Place the encoded value inside the URL hash using the format `#s=...`.
10. Implement `parseShareHash()` to decode and validate incoming greeting links.
11. Create `js/share.js` and integrate:
    - WhatsApp sharing through `https://wa.me/`.
    - Native sharing using `navigator.share`.
    - Clipboard copying using the Clipboard API.
12. Create `js/ui.js` and implement the complete application flow:
    - Sender Card.
    - Receiver Welcome Card.
    - Share Card.
    - Receiver End Card.
13. Capture a thumbnail of the completed fireworks scene at 7.75 seconds.
14. Implement the reciprocal greeting functionality that swaps sender and recipient names.

Privacy architecture:

```
Sender Enters Names
        |
        v
  Input Validation
        |
        v
   JSON Payload
        |
        v
 UTF-8 Encoding
        |
        v
   Base64url
        |
        v
 URL Hash (#s=...)
        |
        v
    Share Link
        |
        v
 Recipient Opens Link
        |
        v
  Decode and Validate
        |
        v
 Receiver Greeting
```

Example generated link:

Plain text

```
https://diwalisky.vercel.app/#s=eyJ2IjoxLCJ0IjoiUHJpeWEiLCJmIjoiVml2ZWsiLCJnIjoiZGl3YWxpIn0
```

Critical privacy distinction: Browsers do not include URL fragments in HTTP requests to the server. Therefore, the recipient and sender names are not transmitted to the hosting server through this sharing mechanism. However, the Base64url encoding is not encryption: anyone who obtains the complete link can decode its contents.

Engineering decision: A versioned URL payload makes the application shareable without requiring a database, user authentication or backend API. Unicode-safe encoding also preserves Tamil and Hindi names.

- • Screenshot (optional): `diwali-sky-step-8.png`
- • Checkpoint: Recipient and sender validation rejects invalid inputs, Tamil and Hindi names survive a complete encode/decode round trip, the generated URL contains a `#s=` payload, and the receiver flow correctly displays the original sender and recipient information.

Step 9: Optimize Rendering, Implement Quality Tiers and Run Automated Tests

- **Instruction:**

Optimize the application for different hardware capabilities and verify that all major functionality works correctly.

1. Update `js/main.js` to integrate `SkyManager`, `ParticlePool`, `FireworksEngine`, `ShowTimeline`, `SoundManager` and `UIManager`.
2. Implement the main rendering loop using `requestAnimationFrame()`.
3. Calculate elapsed frame time and cap delta time at 0.05 seconds.
4. Implement three performance tiers:

Setting	High	Medium	Low
Active particle budget	6,000	2,500	1,500
Lanterns	10	6	3
Embers	40	20	Disabled
Mist bands	3	2	Disabled
Bloom	Enabled	Disabled	Disabled

5. Measure average FPS over two-second intervals.
6. Automatically step down one quality tier when average FPS falls below 45.
7. Prevent automatic quality upgrades during an active show.
8. Implement the developer debugging HUD with live FPS, particle count and rocket count.
9. Add manual Peony, Ring, Willow and Crackle test buttons.
10. Create `js/selftest.js` and execute the automated test suite.
11. Use the browser query parameter `?test=1` to run the tests.
12. Use `?benchmark=1&show=1` to measure animation performance.
13. Inspect the results using Chrome DevTools.

Rendering loop architecture:

JavaScript

```
function loop(timestamp) {

    const deltaTime = Math.min(
        (timestamp - previousTime) / 1000,
        0.05
    );

    previousTime = timestamp;

    skyManager.updateAndDraw(deltaTime);

    fireworks.update(deltaTime);

    particlePool.update(deltaTime);

    show.update(deltaTime);

    particlePool.draw();
```

```
    skyManager.updateBloom();

    requestAnimationFrame(loop);
}
```

Important: This is the rendering pipeline structure; integrate the methods with the actual class interfaces and shared timing state in your implementation.

Testing commands:

Plain text

```
http://localhost:8000/?test=1
```

Plain text

```
http://localhost:8000/?debug=1
```

Plain text

```
http://localhost:8000/?benchmark=1&show=1
```

Performance decision: Do not force every device to render the same number of particles. Adaptive quality management maintains usability on mobile devices while preserving richer effects on capable hardware.

- Screenshot (optional): `diwali-sky-step-9.png`
- Checkpoint: All 43 documented automated tests pass across four test suites, the developer HUD displays live performance information, the benchmark records average FPS, median FPS and 1% low FPS, and performance below 45 FPS triggers the appropriate quality downgrade when quality is not manually locked.

Step 10: Deploy the Application and Publish the Portfolio Project

- **Instruction:**

Prepare the final application for production and publish it using GitHub and Vercel.

1. Verify that the application works on desktop and mobile browsers.
2. Confirm that all assets, JavaScript modules, CSS files and metadata load correctly.
3. Verify the MIT License and project documentation.
4. Initialize Git if it has not already been initialized.
5. Connect the GitHub repository.
6. Push the verified source code to the production branch.
7. Import the GitHub repository into Vercel.
8. Configure Vercel with:
    - Framework Preset: Other.
    - Build Command: None.
    - Output Directory: `.` (project root).
9. Deploy the static website.
10. Verify the production URL and test the sharing workflow using a separate browser session.

Git commands:

Bash

```
git init

git add .

git commit -m "Build Diwali Sky fireworks greeting application"

git branch -M main
```

```
git remote add origin https://github.com/Pradeep-B28/Diwali_Sky.git

git push -u origin main
```

Production deployment architecture:

Plain text

```
Local Development
       |
       v
 Automated Testing
       |
       v
   Git Commit
       |
       v
   GitHub Main
       |
       v
 Vercel Deployment
       |
       v
  Edge Network
       |
       v
 Public Application
```

Production verification:

- Live application: https://diwalisky.vercel.app/ ↗
- GitHub repository: https://github.com/Pradeep-B28/Diwali_Sky ↗

Engineering decision: A static deployment is appropriate because the application has no server-side business logic, database or authentication requirement. Vercel can serve the HTML, CSS, JavaScript and assets directly without a framework build process.

- - Screenshot (optional): `diwali-sky-step-10.png`
- - Checkpoint: The Vercel deployment completes successfully, the production URL returns the application, all assets load without 404 errors, and a generated greeting link opens the correct receiver experience in a separate browser.

5. Expected result

* Demo video link: Pending demo recording

Your app should:

- **Generate Personalized Fireworks:** Accept recipient and sender names and transform the recipient's name into thousands of animated sparks using Canvas 2D and particle physics.
- **Render a Complete Festival Environment:** Display an illuminated South Indian temple skyline, glowing diyas, floating lanterns, atmospheric effects and four distinct firework formations.
- **Produce Real-Time Audio:** Generate synchronized rocket whistles, explosion sounds, crackles and temple bells using the Web Audio API without prerecorded audio files.
- **Support Private Greeting Sharing:** Generate versioned Base64url greeting links, support WhatsApp and native sharing, and reconstruct the receiver experience without storing names on an application database.
- **Deliver an Optimized Responsive Experience:** Support mobile and desktop layouts, adaptive particle budgets, reduced-motion preferences, debugging tools and automated verification with 43 tests.

6. LinkedIn post text

I just built **Diwali Sky**, an interactive personalized fireworks greeting application, using the free project guide from @Wrench Wise.

Turn any person's name into thousands of animated fireworks using JavaScript Canvas, real-time particle physics, procedural Web Audio synthesis and privacy-first URL sharing — all without third-party frameworks or backend dependencies.

Try it: https://diwalisky.vercel.app/ ↗

#WrenchWiseProjects #JavaScript #FrontendDevelopment